

LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

JOBSHEET 11:

LINKED LIST



Aylafada Syakira

TI-1C

254107020116

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG 2026

1. Tujuan Praktikum

Setelah melakukan materi praktikum ini, mahasiswa mampu:

- Membuat struktur data linked list
- Membuat linked list pada program
- Membedakan permasalahan apa yang dapat diselesaikan menggunakan linked list

2. Praktikum

2.1 Pembuatan Single Linked List

1. Pada Project yang sudah dibuat pada minggu sebelumnya, membuat folder baru dengan nama minggul1
2. Menambahkan class-class Mahasiswa06, Node06, SingleLinkedList06, SLLMain06
3. Mengimplementasikan Class Mahasiswa06 sesuai dengan diagram class

```
public class Mahasiswa06 {  
    String nim;  
    String nama;  
    String kelas;  
    double ipk;  
  
    public Mahasiswa06 (String nm, String name, String kls, double ip){  
        nim = nm;  
        nama = name;  
        kelas = kls;  
        ipk = ip;  
    }  
  
    public void tampilInformasi() {  
        System.out.printf(format: "%-10s %-15s %-8s %-5s\n", nama, nim, kelas, ipk);  
    }  
}
```

4. Mengimplementasi class Node seperti gambar

```
public class Node06 {  
    Mahasiswa06 data;  
    Node06 next;  
  
    public Node06(Mahasiswa06 data, Node06 next) {  
        this.data = data;  
        this.next = next;  
    }  
}
```

5. Menambahkan attribute head dan tail pada class SingleLinkedList

```
Node06 head;  
Node06 tail;
```

6. Sebagai langkah berikutnya, akan diimplementasikan method-method yang terdapat pada SingleLinkedList.
7. Menambahkan method isEmpty().

```
boolean isEmpty() {  
    return (head == null);  
}
```

8. Implementasi method untuk mencetak dengan menggunakan proses traverse.

```
public void print() {
    if (!isEmpty()) {
        Node06 tmp = head;
        System.out.print(s: "Isi Linked List:\t");
        while (tmp != null) {
            tmp.data.tampilInformasi();
            tmp = tmp.next;
        }
        System.out.println(x: "");
    } else {
        System.out.println(x: "Linked list kosong");
    }
}
```

9. Mengimplementasikan method addFirst()

```
public void addFirst(Mahasiswa06 input) {
    Node06 ndInput = new Node06(input, next: null);
    if (isEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        ndInput.next = head;
        head = ndInput;
    }
}
```

10. Mengimplementasikan method addLast()

```
public void addLast(Mahasiswa06 input) {
    Node06 ndInput = new Node06 (input, next: null);
    if(isEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        tail.next = ndInput;
        tail = ndInput;
    }
}
```

11. Mengimplementasikan method insertAfter, untuk memasukkan node yang memiliki data input setelah node yang memiliki data keyMengimplementasikan method addLast()

```
public void insertAfter(String key, Mahasiswa06 input) {
    Node06 ndInput = new Node06(input, next: null);
    Node06 temp = head;
    do {
        if (temp.data.nama.equalsIgnoreCase(key)) {
            ndInput.next = temp.next;
            temp.next = ndInput;
            if (ndInput.next == null) {
                tail = ndInput;
            }
            break;
        }
        temp = temp.next;
    } while (temp != null);
}
```

12. Mengimplementasikan method penambahan node pada indeks tertentu.

```
public void insertAt(int index, Mahasiswa06 input) {
    if(index < 0) {
        System.out.println(x: "Indeks salah");
    } else if (index == 0) {
        addFirst(input);
    } else {
        Node06 temp = head;
        for (int i=0; i<index-1; i++){
            temp = temp.next;
        }
        temp.next = new Node06(input, temp.next);
        if (temp.next.next == null) {
            tail = temp.next;
        }
    }
}
```

13. Pada class SLLMain00, membuat fungsi main, kemudian membuat object dari class SingleLinkedList.
14. Membuat empat object mahasiswa dengan nama mhs1, mhs2, mhs3, mhs4 kemudian isi data setiap object melalui konstruktor.

```
Mahasiswa06 mhs1 = new Mahasiswa06(nm: "123", name: "Ayla", kls: "TI-1C", ip: 4);
Mahasiswa06 mhs2 = new Mahasiswa06(nm: "234", name: "Briyani", kls: "SIB-3G", ip: 3);
Mahasiswa06 mhs3 = new Mahasiswa06(nm: "456", name: "Empok", kls: "SIB-1E", ip: 3);
Mahasiswa06 mhs4 = new Mahasiswa06(nm: "567", name: "Uduk", kls: "TI-3G", ip: 2);
```

15. Menambahkan Method penambahan data dan pencetakan data di setiap penambahannya agar terlihat perubahannya.

```
sll.print();
sll.addFirst(mhs4);
sll.print();
sll.addLast(mhs1);
sll.print();
sll.insertAfter(key: "Ayla", mhs3);
sll.insertAt(index: 2, mhs2);
sll.print();
```

2.1.1 Verifikasi Hasil Percobaan

```
Linked list kosong
Isi Linked List:
Uduk      567      TI-3G      2.0

Isi Linked List:
Uduk      567      TI-3G      2.0
Ayla      123      TI-1C      4.0

Isi Linked List:
Uduk      567      TI-3G      2.0
Ayla      123      TI-1C      4.0
Briyani   234      SIB-3G      3.0
Empok     456      SIB-1E      3.0
```

2.1.2 Pertanyaan

1. Mengapa hasil compile kode program di baris pertama menghasilkan “Linked List Kosong”?
 - Karena pada saat method print() pertama kali dipanggil, linked list belum memiliki node. Nilai variable head masih null, sehingga method isEmpty mengembalikan nilai true
2. Jelaskan kegunaan variable temp secara umum pada setiap method!
 - Digunakan sebagai pointer, mencari data tertentu, nilai head agar tidak berubah
3. Lakukan modifikasi agar data dapat ditambahkan dari keyboard!

```
public class SLLMain06 {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        SingleLinkedList06 sll = new SingleLinkedList06();

        System.out.println(x: "Masukkan NIM: ");
        String nim = sc.nextLine();
        System.out.println(x: "Masukkan Nama: ");
        String nama = sc.nextLine();
        System.out.println(x: "Masukkan Kelas: ");
        String kelas = sc.nextLine();
        System.out.println(x: "Masukkan IPK: ");
        double ipk = sc.nextDouble();

        Mahasiswa06 mhs = new Mahasiswa06(nim, nama, kelas, ipk);
        sll.addFirst(mhs);
        sll.print();
    }
}
```

2.2 Modifikasi Elemen pada Single Linked List

2.2.1 Langkah-langkah Percobaan

1. Mengimplementasikan method untuk mengakses data dan indeks pada linked list
2. Menambahkan method untuk mendapatkan data pada indeks tertentu pada class Single Linked List

```
public void getData(int index) {
    Node06 tmp = head;
    for (int i = 0; i < index; i++) {
        tmp = tmp.next;
    }
    tmp.data.tampilInformasi();
}
```

3. Mengimplementasikan method indexOf

```
public int indexOf(String key) {
    Node06 tmp = head;
    int index = 0;
    while (tmp != null && tmp.data.nama.equalsIgnoreCase(key)) {
        tmp = tmp.next;
        index++;
    }

    if (tmp == null) {
        return -1;
    } else {
        return index;
    }
}
```

4. Menambahkan method removeFirst pada class SingleLinkedList

```
public void removeFirst() {
    if (isEmpty()) {
        System.out.println(x: "Linked List masih kosong, tidak dapat dihapus");
    } else if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
    }
}
```

5. Menambahkan method untuk menghapus data pada bagian belakang pada class SingleLinkedList

```
public void removeLast() {
    if (isEmpty()) {
        System.out.println(x: "Link List masih kosong, tidak dapat dihapus");
    } else if (head == tail) {
        head = tail = null;
    } else {
        Node06 temp = head;
        while (temp.next != tail) {
            temp = temp.next;
        }
        temp.next = null;
        tail = temp;
    }
}
```

6. Sebagai langkah berikutnya, akan diimplementasikan method remove

```

public void remove (String key) {
    if (isEmpty()) {
        System.out.println(x: "Linked List masih kosong, tidak dapat dihapus");
    } else {
        Node06 temp = head;
        while (temp != null) {
            if ((temp.data.nama.equalsIgnoreCase(key)) && (temp == head)) {
                this.removeFirst();
                break;
            } else if (temp.data.nama.equalsIgnoreCase(key)) {
                temp.next = temp.next.next;
                if (temp.next == null) {
                    tail = temp;
                }
                break;
            }
            temp = temp.next;
        }
    }
}
}

```

7. Mengimplementasikan method untuk menghapus node dengan menggunakan index.

```

public void removeAt(int index) {
    if (index == 0) {
        removeFirst();
    } else {
        Node06 temp = head;
        for (int i=0; i<index-1; i++) {
            temp = temp.next;
        }

        temp.next = temp.next.next;
        if (temp.next == null) {
            tail = temp;
        }
    }
}
}

```

8. Kemudian, melakukan pengaksesan dan penghapusan data di method main pada class SLLMain dengan menambahkan kode berikut

```

System.out.println(x: "data index 1: ");
sll.getData(index: 1);

System.out.println("data mahasiswa an Bimon berada pada index : " +sll.indexOf(key: "bimon"));
System.out.println();

sll.removeFirst();
sll.removeLast();
sll.print();
sll.removeAt(index: 0);
sll.print();

```

2.2.2 Verifikasi Hasil Percobaan

```
data index 1:
Ayla      123          TI-1C    4.0
data mahasiswa an Ayla berada pada index : 0

Isi Linked List:
Ayla      123          TI-1C    4.0
Briyani   234          SIB-3G   3.0

Isi Linked List:
Briyani   234          SIB-3G   3.0
```

2.2.3 Pertanyaan

1. Mengapa digunakan keyword break pada fungsi remove? Jelaskan!
 - Break digunakan untuk memberhentikan perulangan setelah data sudah ditemukan. Tanpa break, program akan tetap melanjutkan pencarian hingga akhir linked list meskipun node yang dimaksud sudah dihapus yang membuat proses tidak efisien
2. Jelaskan kegunaan kode dibawah pada method remove

```
1 temp.next = temp.next.next;
2 if (temp.next == null) {
3     tail = temp;
4 }
```

- Digunakan untuk memperbarui nilai tail Ketika tail yang terakhir di hapus

3. Tugas

Buatlah implementasi program antrian layanan unit kemahasiswaan sesuai dengan berikut ini:

- a. Implementasi antrian menggunakan Queue berbasis Linked List!
- b. Program merupakan proyek baru bukan modifikasi dari percobaan
- c. Ketika seorang mahasiswa akan mengantri, maka dia harus mendaftarkan datanya
- d. Cek antrian kosong, Cek antrian penuh, Mengosongkan antrian.
- e. Menambahkan antrian
- f. Memanggil antrian
- g. Menampilkan antrian terdepan dan antrian paling akhir
- h. Menampilkan jumlah mahasiswa yang masih mengantre

1. Class mahasiswa06

```
public class mahasiswa06 {
    String nim;
    String nama;
    String prodi;

    public mahasiswa06(String nim, String nama, String prodi) {
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;
    }

    public void tampil() {
        System.out.printf(format: "%-10s %-15s %-10s\n", nim, nama, prodi);
    }
}
```

2. Class node06

```
public class node06 {
    mahasiswa06 data;
    node06 next;

    public node06(mahasiswa06 data) {
        this.data = data;
        this.next = null;
    }
}
```

3. Class queueLinkedList06

```
public class queueLinkedList06 {
    node06 front;
    node06 rear;
    int size;

    public queueLinkedList06() {
        front = null;
        rear = null;
        size = 0;
    }

    public boolean isEmpty() {
        return front == null;
    }

    public boolean isFull() {
        return false;
    }

    public void clear() {
        front = null;
        rear = null;
        size = 0;
    }
}
```

```

public void enqueue(mahasiswa06 data) {
    node06 temp = new node06(data);
    if(isEmpty()) {
        front = rear = temp;
    } else {
        rear.next = temp;
        rear = temp;
    }
    size++;
}

public mahasiswa06 dequeue() {
    if(isEmpty()) {
        return null;
    }
    mahasiswa06 keluar = front.data;
    front = front.next;
    if(front == null) {
        rear = null;
    }
    size--;
    return keluar;
}

public void peekFront() {
    if(isEmpty()) {
        System.out.println(x: "Antrian kosong");
    } else {
        System.out.println(x: "Antrian terdepan: ");
        front.data.tampil();
    }
}

public void peekRear() {
    if(isEmpty()) {
        System.out.println(x: "Antrian kosong");
    } else {
        System.out.println(x: "Antrian paling akhir: ");
        rear.data.tampil();
    }
}

public void jumlahAntrian() {
    System.out.println("Jumlah antrian: " +size);
}

public void print() {
    if(isEmpty()) {
        System.out.println(x: "Antrian kosong");
        return;
    }
    node06 temp = front;
    System.out.println(x: "\nDaftar antrian");
    while(temp != null) {
        temp.data.tampil();
        temp = temp.next;
    }
}

```

4. Class main06

```
public class main06 {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        queueLinkedList06 antrian = new queueLinkedList06();
        int pilih;

        do {
            System.out.println(x: "---- ANTRIAN MAHASISWA ----");
            System.out.println(x: "1. Tambah Antrian");
            System.out.println(x: "2. Panggil Antrian");
            System.out.println(x: "3. Lihat Antrian Paling Depan");
            System.out.println(x: "4. Lihat Antrian Paling Belakang");
            System.out.println(x: "5. Jumlah Antrian");
            System.out.println(x: "6. Tampilkan Semua");
            System.out.println(x: "7. Kosongkan Antrian");
            System.out.println(x: "0. Keluar");
            System.out.print(s: "Pilih: ");
            pilih = sc.nextInt();
            sc.nextLine();

            switch(pilih) {
                case 1:
                    System.out.print(s: "NIM: ");
                    String nim = sc.nextLine();
                    System.out.print(s: "Nama: ");
                    String nama = sc.nextLine();
                    System.out.print(s: "Prodi: ");
                    String prodi = sc.nextLine();
                    antrian.enqueue(new mahasiswa06(nim, nama, prodi));
                    break;
                case 2:
                    mahasiswa06 keluar = antrian.dequeue();
                    if(keluar != null) {
                        System.out.print(s: "Dipanggil: ");
                        keluar.tampil();
                    }
                    break;
                case 3:
                    antrian.peekFront();
                    break;
                case 4:
                    antrian.peekRear();
                    break;
                case 5:
                    antrian.jumlahAntrian();
                    break;
                case 6:
                    antrian.print();
                    break;
                case 7:
                    antrian.clear();
                    System.out.println(x: "Antrian Dikosongkan");
                    break;
            }

        } while (pilih != 0);
    }
}
```

3.1 Verifikasi Hasil

```
---- ANTRIAN MAHASISWA ----
1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Paling Depan
4. Lihat Antrian Paling Belakang
5. Jumlah Antrian
6. Tampilkan Semua
7. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM: 1234
Nama: Ayla
Prodi: TI
---- ANTRIAN MAHASISWA ----
1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Paling Depan
4. Lihat Antrian Paling Belakang
5. Jumlah Antrian
6. Tampilkan Semua
7. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM: 2345
Nama: Fada
Prodi: TI
---- ANTRIAN MAHASISWA ----
1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Paling Depan
4. Lihat Antrian Paling Belakang
5. Jumlah Antrian
6. Tampilkan Semua
7. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM: 4567
Nama: Syakira
Prodi: SIB

---- ANTRIAN MAHASISWA ----
1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Paling Depan
4. Lihat Antrian Paling Belakang
5. Jumlah Antrian
6. Tampilkan Semua
7. Kosongkan Antrian
0. Keluar
Pilih: 6
Daftar antrian
1234      Ayla      TI
2345      Fada      TI
4567      Syakira    SIB
---- ANTRIAN MAHASISWA ----
1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Paling Depan
4. Lihat Antrian Paling Belakang
5. Jumlah Antrian
6. Tampilkan Semua
7. Kosongkan Antrian
0. Keluar
Pilih: 3
Antrian terdepan:
1234      Ayla      TI
---- ANTRIAN MAHASISWA ----
1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Paling Depan
4. Lihat Antrian Paling Belakang
5. Jumlah Antrian
6. Tampilkan Semua
7. Kosongkan Antrian
0. Keluar
Pilih: 4
Antrian paling akhir:
4567      Syakira    SIB
```

```
---- ANTRIAN MAHASISWA ----
1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Paling Depan
4. Lihat Antrian Paling Belakang
5. Jumlah Antrian
6. Tampilkan Semua
7. Kosongkan Antrian
0. Keluar
Pilih: 5
Jumlah antrian: 3
```

```
---- ANTRIAN MAHASISWA ----
1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Paling Depan
4. Lihat Antrian Paling Belakang
5. Jumlah Antrian
6. Tampilkan Semua
7. Kosongkan Antrian
0. Keluar
Pilih: 2
```

```
Dipanggil: 1234      Ayla
---- ANTRIAN MAHASISWA ----
1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Paling Depan
4. Lihat Antrian Paling Belakang
5. Jumlah Antrian
6. Tampilkan Semua
7. Kosongkan Antrian
0. Keluar
Pilih: 6
```

```
Daftar antrian
2345      Fada      TI
4567      Syakira   SIB
```

```
---- ANTRIAN MAHASISWA ----
1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Paling Depan
4. Lihat Antrian Paling Belakang
5. Jumlah Antrian
6. Tampilkan Semua
7. Kosongkan Antrian
0. Keluar
```

```
TI pilih: 7
Antrian Dikosongkan
```

```
---- ANTRIAN MAHASISWA ----
1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Paling Depan
4. Lihat Antrian Paling Belakang
5. Jumlah Antrian
6. Tampilkan Semua
7. Kosongkan Antrian
0. Keluar
```

```
Pilih: 0
PS D:\college\semester 2\PASD>
```